

Taking over an unknown project: where to start

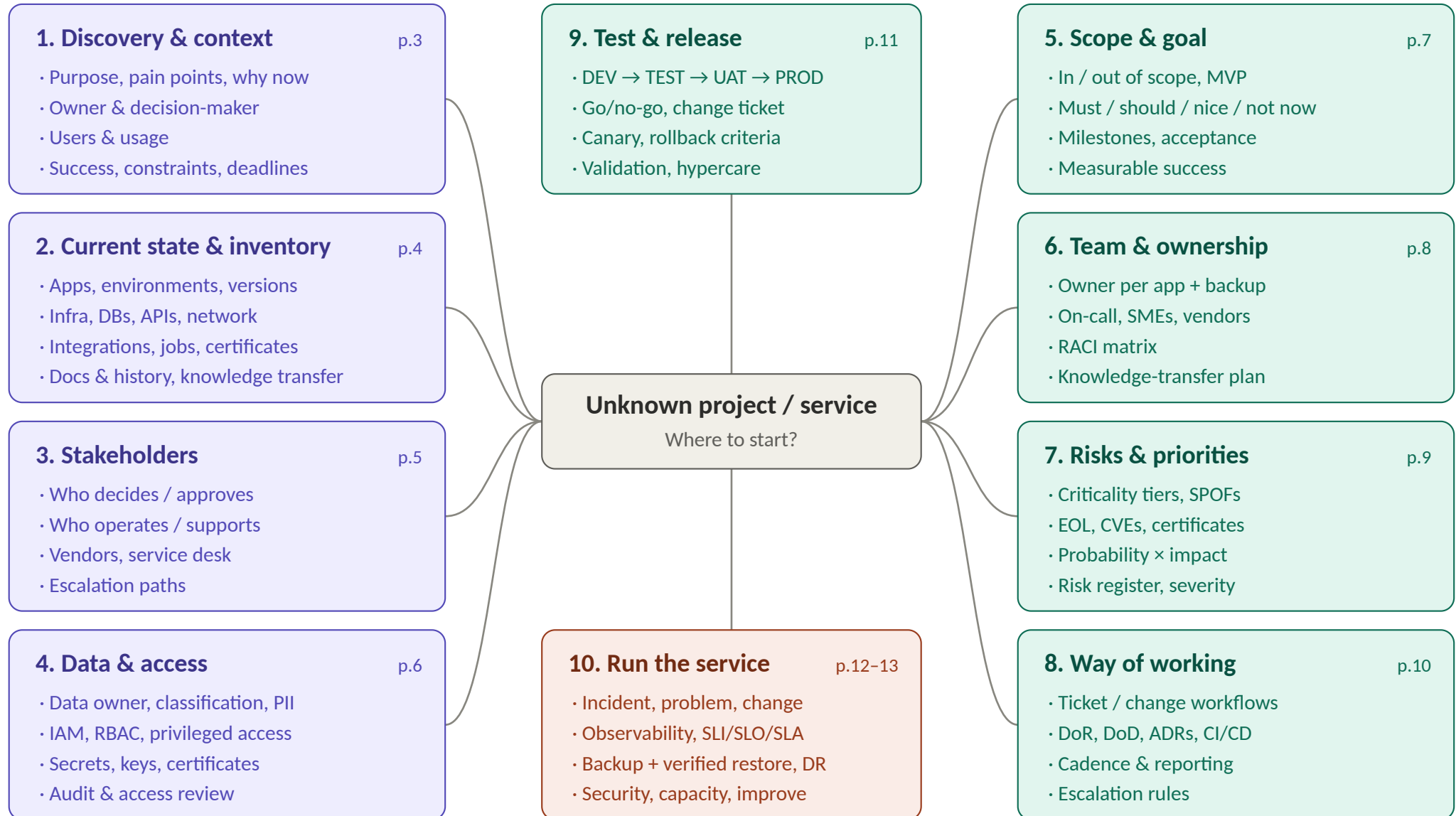
Stabilize first. Then understand (1–4), shape and deliver (5–9), and run (10). Each box links to its detailed page.

 Prerequisite Understand first
 Shape & deliver Run

0. Stabilize — prerequisite

p.2

Is anything burning? Production outage · major / security incidents · failed backups · expiring certificates · capacity · critical deadlines · business risk



Objective Before any analysis, confirm that nothing is burning. Stabilize first, understand second.

ACTIONS

- Check production status and user-facing availability
- Review open P1/P2 (highest-priority) incidents and recurring alerts
- Ask security about open security incidents or findings
- Check last backup results and the date of the last successful restore
- Scan certificate expiry dates (30 / 60 / 90 days)
- Check disk, CPU, memory, database and queue capacity trends
- Check licence and vendor support contract expiry
- List hard deadlines: releases, audits, regulatory dates, end of support
- Set up a temporary escalation contact list and incident channel
- Decide whether to freeze non-essential changes until the picture is clear

CHECK

- ✓ Is production healthy right now?
- ✓ Any active major or security incident?
- ✓ Did the last backups succeed?
- ✓ Any certificate expiring within 30 days?
- ✓ Any filesystem or DB above 85% full?
- ✓ Any business deadline in the next 2–4 weeks?

DELIVERABLES

- ▶ Immediate health-check report (1 page)
- ▶ Burning-issues list with owner and due date
- ▶ Temporary escalation contacts
- ▶ Change-freeze decision (yes / no, until when)

RED FLAGS

- ! Nobody can say whether production is healthy
- ! Backups failing silently for days
- ! Certificates expiring this week
- ! Disks filling up without alerts
- ! Open P1 incident with no owner
- ! A go-live or audit next week that nobody mentioned

EXIT CRITERIA

No unowned critical issue. Every burning item has an owner, an action and a date. You know whom to call when something breaks.

EXAMPLE

Day-1 check finds the TLS certificate on the customer API gateway expires in 9 days and the main database backup has failed for 6 nights. Both go to the burning-issues list with owners: certificate renewal raised as an urgent change; backup fixed and confirmed by a test restore.

HEALTH-CHECK TRAFFIC LIGHT

Area	What to check	Status
Production	Availability, error rate, user complaints	○○○
Incidents	Open P1/P2, repeating incidents	○○○
Security	Open incidents, critical vulnerabilities (CVEs)	○○○
Backups	Last success, last verified restore	○○○
Certificates	Expiry < 30 days	○○○
Capacity	Disk, DB, CPU, memory, queues	○○○
Deadlines	Releases, audits, end of support	○○○
Contracts	Licences, vendor support	○○○

Objective Understand why the system exists, what is expected of it, and what has already been decided.

ACTIONS

- Document the business purpose: which process the system supports and who depends on it
- Identify the owner who takes final decisions
- Identify user groups, volumes, peak hours and business calendar
- Collect current pain points from tickets, incidents and user feedback
- Capture expected outcomes and how success will be measured
- List constraints: budget, people, technology, regulation, contracts
- Collect known commitments: SLAs (service level agreements), promises to the business, vendor contracts
- Collect existing deadlines and fixed dates
- Record decisions already made, by whom and why
- Start an assumptions & unknowns log and review it weekly

CHECK

- ✓ Business purpose confirmed by the owner in writing
- ✓ Success defined as a measurable metric
- ✓ Constraints and deadlines confirmed
- ✓ Assumptions clearly marked as assumptions, not facts

RED FLAGS

- ! No clear owner, or several people claiming ownership
- ! Success described only as “it works”
- ! Commitments made to the business that nobody wrote down
- ! Conflicting goals between stakeholders
- ! Decisions made verbally, with no record

DELIVERABLES

- ▶ Discovery summary (1–2 pages)
- ▶ Assumptions & unknowns log
- ▶ Decision log
- ▶ List of commitments and deadlines

EXIT CRITERIA

Purpose, owner, users, success metric, constraints and deadlines are documented and confirmed by the owner. Open unknowns are logged with an owner.

EXAMPLE

Internal loan-processing application: processes ~2,000 applications per day for branch staff. Pain point: the nightly batch overruns into business hours twice a week. Expected outcome: batch finished by 06:00 on 99% of days. Constraint: vendor contract ends in 8 months. Decision already made: migration to Azure in 2027.

ASSUMPTIONS & UNKNOWNNS LOG (EXAMPLE)

ID	Item	Type	Owner	Validate by	Status
A-01	Batch volume will not grow more than 10% this year	Assumption	Business owner	15 Oct	Open
U-02	Who owns the SFTP interface to the partner bank?	Unknown	Service owner	10 Oct	Open
D-03	Migration to Azure in 2027 (approved by CIO)	Decision	CIO	—	Confirmed

Objective Understand what actually exists before changing anything. Knowledge transfer starts here.

ACTIONS

- Discover applications and components; compare documentation, CMDB (configuration management database) and reality
- Identify environments DEV / TEST / UAT (user acceptance testing) / PROD and how they differ
- Record runtime/platform, operating systems, middleware, databases
- Identify APIs, integrations, external services and third-party vendors
- Map network dependencies: DNS, load balancers, firewalls, proxies
- Find certificates: location, issuer, expiry
- Find scheduled jobs and batch processing: schedule, runtime, dependencies
- Map data flows between systems
- Review existing CI/CD (continuous integration / delivery) and how deployment is done today
- Review monitoring and logging coverage
- Collect versions and end-of-life (EOL) dates
- Collect existing docs, runbooks, known issues and incident history

CHECK

- ✓ Inventory matches reality, not just the documentation
- ✓ Every component has an owner
- ✓ Every PROD dependency is known
- ✓ EOL components identified
- ✓ You can describe how a deployment happens today

DELIVERABLES

- ▶ Application inventory
- ▶ Environment matrix
- ▶ Dependency map & data flows
- ▶ Architecture diagram
- ▶ Technology / version inventory with EOL dates
- ▶ Certificate and scheduled-job list
- ▶ Documentation gap list

RED FLAGS

- ! Unknown production dependencies
- ! Unsupported / EOL software
- ! No architecture documentation
- ! Unknown owner
- ! Manual deployment known by one person
- ! No monitoring
- ! Hard-coded IPs or credentials

EXIT CRITERIA

We can explain the system architecture, dependencies, owners and operational model. The inventory has been reviewed with the team.

EXAMPLE

Inventory reveals a nightly cron job on a “decommissioned” VM that sends payment files to a partner bank via SFTP. Nobody had listed it. It is added to the dependency map, given an owner, monitored, and scheduled for migration.

ENVIRONMENT MATRIX (EXAMPLE)

	DEV	TEST	UAT	PROD
Hosting	Azure dev sub.	Azure test sub.	On-prem VMware	On-prem VMware + disaster-recovery site
App version	4.3-SNAPSHOT	4.2.1	4.2.0	4.1.3
Data	Synthetic	Masked	Masked	Real (personal data)
Deployment	CI/CD auto	CI/CD auto	CI/CD, approval	Manual script ✗
Monitoring	None	Basic	Basic	Dynatrace + logs
Owner	Dev team	Dev team	App support	App support

Objective Know who decides, who operates, who supports, who approves and who must be informed — before you need them in an incident.

ACTIONS

- List all roles: product/business owner, service owner, application owner, infrastructure, network, database, cloud/platform, service desk, security/compliance, vendors, change & release management, end users
- Name a person and a deputy for each role
- Classify each role: decides / operates / supports / approves / informed
- Define escalation paths for incidents and for decisions
- Agree communication channels and cadence per group
- Confirm vendor contacts, support contracts and response times

CHECK

- ✓ Every role has a named person and a deputy
- ✓ Escalation contacts reachable (24/7 where needed)
- ✓ Service desk knows how to route tickets
- ✓ Vendor support contract valid

RED FLAGS

- ! Owner unknown or “the whole department”
- ! Vendor support expired
- ! Tickets routed to someone who left
- ! No after-hours contact
- ! Security involved only at the end

DELIVERABLES

- ▶ Stakeholder map
- ▶ Contact list with deputies
- ▶ Escalation matrix
- ▶ Communication plan

EXIT CRITERIA

Stakeholder map and escalation matrix agreed and published. Everyone knows whom to call.

EXAMPLE

PROD database outage at 02:00: service desk calls the on-call application engineer (L2) → DBA on call (L3) within 30 min → service owner informed after 60 min without resolution → severity-1 ticket with the database vendor.

STAKEHOLDER MAP & ESCALATION PATH

Role	Decides	Operates	Supports	Approves	Informed
Business / product owner	•			•	
Service owner	•			•	
Application owner / support		•	•		
Infra, network, DB, cloud		•	•		
Service desk (L1)			•		
Security / compliance				•	•
Change & release mgmt				•	•
Vendors			•		
End users					•

Service desk (L1) → App support (L2) → Specialists / vendor (L3) → Service owner → Management

Objective Know who owns the data, how sensitive it is and who can access what — and make sure the team has the access it needs.

ACTIONS

- Identify data owner and data steward per dataset
- Classify data (public / internal / confidential / restricted); identify PII (personally identifiable information)
- Document retention, deletion, encryption (at rest / in transit), backup location
- Identify regulatory requirements (e.g. GDPR, DORA, PCI DSS, internal policy)
- Review IAM (identity & access management) and RBAC (role-based access control): roles, groups, least privilege
- List service accounts and privileged accounts, and who uses them
- Locate secrets: password vault, API keys, certificates, SSH keys, DB credentials
- Review database access and production access; test break-glass access
- Verify audit logging: who did what, when
- Schedule periodic access review (e.g. quarterly)
- Request missing access for the team

CHECK

- ✓ No shared personal accounts
- ✓ Secrets in a vault, not in code or scripts
- ✓ Break-glass tested and logged
- ✓ Leavers removed
- ✓ PII masked in non-PROD and in logs
- ✓ Audit logs retained per policy

RED FLAGS

- ! Passwords in scripts or wiki
- ! Shared admin accounts
- ! Service-account password never rotated
- ! Former employees still have access
- ! Unmasked production data in TEST
- ! No audit trail on PROD

DELIVERABLES

- ▶ Access matrix
- ▶ Data ownership matrix
- ▶ Privileged access list
- ▶ Missing-access list
- ▶ Secrets inventory (location, owner, rotation, expiry)

EXIT CRITERIA

Data ownership and classification documented, access matrix agreed, the team has the access it needs, privileged access under control.

EXAMPLE

Review finds the database password of a reporting service in a plain-text config file in Git, unchanged for 4 years. It is moved to the vault, rotated, removed from repository history and added to the rotation schedule.

ACCESS MATRIX (EXAMPLE)

Role / account	DEV	TEST	UAT	PROD	Approver
Developer	Full	Full	Read	None (break-glass only)	App owner
Application support	Read	Read	Deploy	Operate, no data access	Service owner
DBA	Admin	Admin	Admin	Admin via PAM (privileged access mgmt), logged	Head of infra
svc-reporting (service acc.)	—	—	—	DB read, vaulted, rotated 90 d	App owner

Objective Turn discovery into an agreed scope: what we deliver, in which order, by when, and how success is measured.

ACTIONS

- Define in scope and out of scope
- Set business priorities and technical priorities
- Define the MVP (minimum viable product): the smallest version that solves the core problem
- Prioritise with Must / Should / Nice / Not now
- Identify dependencies on other teams, projects and vendors
- Confirm constraints: budget, deadline, people, change windows
- Define acceptance criteria per deliverable
- Define milestones
- Define success: baseline + target metric
- Write a one-page charter and get sign-off; put scope under change control

CHECK

- ✓ Owner has signed off the scope
- ✓ Every must-have has acceptance criteria
- ✓ Milestones realistic vs team capacity
- ✓ Success metric has a baseline

RED FLAGS

- ! Everything is “must have”
- ! No out-of-scope list
- ! Deadline fixed but scope and team not
- ! Success cannot be measured
- ! Hidden dependencies on other projects

DELIVERABLES

- ▶ Charter (1 page)
- ▶ Scope statement: in / out
- ▶ Prioritised backlog
- ▶ Milestone plan
- ▶ Acceptance criteria

EXIT CRITERIA

Scope, priorities, milestones and success metric agreed and signed off. Scope changes go through change control.

EXAMPLE

Monitoring modernisation. Must: OpenTelemetry for 3 critical services, alerting with runbooks. Should: SLO (service level objective) dashboards. Nice: anomaly detection. Not now: legacy mainframe. M1 (6 weeks): one service end-to-end. Success: mean time to restore (MTTR, defined in section 10) from 4 h to 1 h.

PRIORITISATION

Must have

Without it the delivery fails or is not compliant.

Should have

Important, but a workaround exists for now.

Nice to have

Adds value; first to drop when time is short.

Not now

Explicitly out of this phase; recorded, not forgotten.

Scope, time and people move together: if one changes, at least one other must change too.

Objective Every component, process and decision has a named owner, and no knowledge depends on one person.

ACTIONS

- Assign a service owner per application
- Name technical, operational, backup (deputy) and escalation owners
- Define on-call responsibility and rota
- Identify subject matter experts (SMEs) and their availability
- Clarify vendor responsibility (contract scope, SLA)
- Clarify shared responsibilities with platform, network and DB teams
- Build a RACI (responsible, accountable, consulted, informed) for key activities — exactly one A per activity
- Decide dedicated core team vs ad hoc specialists; what to delegate
- Find single points of knowledge, unowned components and responsibility gaps
- Plan knowledge transfer: document → shadow → reverse shadow → validate

CHECK

- ✓ No component without an owner
- ✓ Every owner has a backup
- ✓ On-call covers the required hours
- ✓ Each critical area known by at least 2 people

DELIVERABLES

- ▶ Ownership matrix
- ▶ RACI
- ▶ On-call rota
- ▶ Knowledge-transfer plan
- ▶ Skills matrix (bus factor)

RED FLAGS

- ! “Only one person knows how this works”
- ! Component owned by “everyone”
- ! Vendor and internal team each think the other owns it
- ! On-call without runbooks
- ! Key person leaving soon

EXIT CRITERIA

Ownership matrix and RACI agreed. No critical single point of knowledge without an active knowledge-transfer plan.

EXAMPLE

Only one engineer knows the payment-batch restart procedure and leaves in 6 weeks. KT plan: runbook written (week 1), shadowing (weeks 2–3), reverse shadowing (weeks 4–5), second engineer restarts alone in TEST (week 6).

RACI (EXAMPLE)

Activity	Service owner	App support	Infra	Vendor
PROD deployment	A	R	C	C
P1 incident resolution	A	R	R	C
OS patching	I	C	A/R	—
Certificate renewal	I	A/R	C	—
Vendor escalation	A	R	I	R

R = does the work · A = accountable, only one · C = consulted · I = informed

Objective Find what can hurt the service most, rank it, and work on the highest risks first.

ACTIONS

- Classify each application by criticality: what happens if it is down for 2 hours?
- Identify single points of failure (SPOFs): infrastructure, people, vendors
- Check unsupported software, EOL components, security vulnerabilities
- Check expiring certificates and capacity limits
- Check missing monitoring, missing backups, missing restore tests
- List manual operational procedures
- Assess vendor dependency, knowledge dependency and technical debt
- Rate probability × impact → risk severity (Critical / High / Medium / Low); assign owner and mitigation
- Spike (test / prototype) the riskiest unknown first
- Review the risk register weekly

CHECK

- ✓ Every risk has owner, mitigation and date
- ✓ Tier-1 services have monitoring and a verified restore
- ✓ Accepted risks formally approved

DELIVERABLES

- Criticality classification
- Risk register
- Top-10 risks by severity for management
- Mitigation plan

RED FLAGS

- ! Risks with no owner
- ! “Accepted” risks nobody approved
- ! Register not updated for months
- ! Critical service on EOL OS
- ! SPOF in PROD with no plan

EXIT CRITERIA

Risk register agreed with the owner; top risks have funded mitigation; priorities are clear to the team.

EXAMPLE

Tier-1 core banking interface runs on a single VM with RHEL 7 (end of maintenance) and has no restore test in 12 months: three risks, one service — top of the register.

RISK REGISTER (EXAMPLE) & RISK SEVERITY

Risk	Impact	Probability	Severity	Owner	Mitigation	Status
Main DB on single VM (SPOF)	High	Medium	High	DBA lead	Add replica, test failover	In progress
RHEL 7 end of maintenance	High	High	Critical	Infra	Upgrade plan, Q1	Open
Only 1 person knows batch restart	High	Medium	High	Service owner	KT plan + runbook	In progress
No restore test for 12 months	High	High	Critical	App support	Verified restore by 15 Oct	Open
API gateway cert expires in 20 days	High	Low	Medium	Platform	Renew + expiry alert	Done

Impact ↓ · Prob. →	Low	Medium	High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

RISK SEVERITY

Critical / High / Medium / Low

Something that *may* happen · probability × impact · risk register

INCIDENT PRIORITY

P1 / P2 / P3 / P4

Something that *is* happening · impact × urgency · section 10

Objective Define how work enters, flows, gets approved and is communicated, so the team works predictably.

ACTIONS

- Define workflows: ticket, incident, change, release (states and owners)
- Code review rules and Git branching (e.g. trunk-based, short-lived branches)
- CI/CD: build, test, scan and deploy automatically
- Change approval: standard / normal / emergency
- Documentation rules: where, format, who updates
- Communication channels: chat, e-mail, incident bridge
- Escalation rules
- Meeting cadence and reporting (key performance indicators, SLOs, risks)
- Backlog management: one backlog, one prioritisation owner
- Agree Definition of Ready and Definition of Done
- Record architecture decisions as ADRs (Architecture Decision Records)

CHECK

- ✓ No work without a ticket
- ✓ DoD applied to every item
- ✓ Decisions traceable
- ✓ Everyone knows where work comes in

DELIVERABLES

- ▶ Working agreement
- ▶ Workflow diagrams
- ▶ DoR / DoD
- ▶ ADR repository
- ▶ Meeting & reporting calendar

RED FLAGS

- ! Work arrives via chat or corridor
- ! Changes without tickets
- ! Decisions only in meetings
- ! No code review
- ! Status report nobody reads

EXIT CRITERIA

Working agreement published and used; all work flows through the defined workflows.

EXAMPLE

Mon backlog refinement · daily 15-min stand-up · Wed ops review (incidents, alerts, capacity) · Fri demo · monthly service review with the owner (SLOs, risks, costs).

DOR / DOD AND ADR

Definition of Ready

- Clear goal and acceptance criteria
- Dependencies known
- Sized and prioritised
- Access / environments available

Definition of Done

- Code reviewed, tests pass in CI
- Deployed to TEST/UAT, validated
- Monitoring & alerts updated
- Runbook / docs updated

ADR = Architecture Decision Record — one short file per decision: *number & title · status (proposed / accepted / superseded) · context · decision · alternatives considered · consequences · date & owner*. Example: ADR-007: Use managed PostgreSQL instead of a self-hosted VM.

New → Ready → In progress → Review → Done

Objective Every change reaches production in a controlled, tested and reversible way.

ACTIONS

- Promote the same artefact through DEV → TEST → UAT → PROD
- Unit and integration tests automatically in CI
- End-to-end (E2E), regression, performance and security testing
- UAT sign-off by the business
- Change ticket with plan, risk and rollback; book maintenance window
- Release checklist and go/no-go decision
- Deploy with pilot group / canary / blue-green
- Smoke test and deployment validation
- Post-deployment validation: metrics, logs, business check
- Hypercare: 1–2 weeks of enhanced monitoring

CHECK

- ✓ The artefact deployed is the one tested in UAT
- ✓ Rollback tested
- ✓ Monitoring ready before go-live
- ✓ Support team briefed

DELIVERABLES

- ▶ Test results & UAT sign-off
- ▶ Change ticket
- ▶ Release checklist & notes
- ▶ Rollback plan
- ▶ Hypercare report

RED FLAGS

- ! Different build in PROD vs UAT
- ! Rollback “not possible”
- ! No smoke tests
- ! Friday-evening deploy without support
- ! Real PII in TEST

EXIT CRITERIA

Change closed, post-deployment validation passed, hypercare finished, handed over to operations.

EXAMPLE

Client portal 4.2: canary to 5% of users for 2 h while error rate and latency are watched; error rate stays < 0.5% → 25% → 100%.
Hypercare 10 days.

RELEASE FLOW, APPROVALS AND FAILURE HANDLING



Who approves

- Business owner — UAT sign-off
- Change manager / Change Advisory Board (CAB) — change ticket
- Service owner — go / no-go
- Release manager — coordinates

If deployment fails

- Rollback criteria met (smoke test fails, error rate above threshold, window exceeded)
- Roll back / switch back (blue-green, feature flag)
- Verify service restored, inform stakeholders
- Incident if users affected; review before next attempt

Objective Keep the service healthy, detect problems before users do, recover fast, and learn from every failure. Two core questions: “How do we know the service is broken?” and “How do we determine why it is broken?”

INCIDENT MANAGEMENT



- Goal: restore service as fast as possible; workaround first, root cause later
- **Incident priority** from impact × urgency: P1 Critical · P2 High · P3 Medium · P4 Low
- Major incident: bridge, comms lead, updates every 30 min
- **MTTR** (Mean Time To Restore) — average time to restore the service after an incident. Target: Tier-1 from 120 to 45 min

PROBLEM MANAGEMENT



- Triggered by P1/P2 or repeating incidents
- RCA (root cause analysis): timeline, 5 whys, contributing factors
- Workaround recorded in Known Error Database (KEDB)
- Permanent fix delivered via change

CHANGE MANAGEMENT

- Standard (pre-approved) · normal (CAB) · emergency
- Who may change PROD is defined and enforced
- Change calendar, freeze periods, maintenance windows
- Every incident checked against recent changes

ON-CALL & ESCALATIONS

- Rota with primary and secondary
- Response times per priority (e.g. P1 15 min, P2 1 h)
- Escalation matrix: L1 → L2 → L3 / vendor → owner
- Handover notes at every shift change

RUNBOOKS & KNOWLEDGE BASE

- One runbook per alert: symptoms, checks, fix, escalation
- Operational procedures: restart, failover, deploy, restore
- Troubleshooting guides, KEDB, vendor docs
- Owned, versioned, reviewed after every incident

POST-INCIDENT REVIEWS (PIR)

- Blameless, within 5 working days of a P1/P2
- Timeline, impact, root cause, what went well
- Actions with owner and date in the backlog
- Update runbooks, alerts and KEDB

EXAMPLE

09:12 synthetic check on the payment API fails → 09:15 on-call acknowledges → dashboard shows DB connection pool exhausted → runbook: recycle pool, scale out → service restored 09:34 → problem record: connection leak introduced in release 4.2 → KEDB entry until the fix ships via normal change → PIR actions: pool-saturation alert, load test in pipeline.

CHECK

- ✓ Every Tier-1 alert has a runbook and an owner
- ✓ On-call reachable within agreed response time
- ✓ Last P1 has a closed PIR with actions
- ✓ No PROD change without a change record
- ✓ KEDB entries have a planned permanent fix

RED FLAGS

- ! Users report outages before monitoring does
- ! Same incident repeats every month
- ! PROD changed outside change process
- ! On-call without runbooks or escalation contacts
- ! PIR actions never closed

OBSERVABILITY

- Metrics, logs, traces (e.g. OpenTelemetry)
- Health checks and synthetic monitoring
- Dashboards per service incl. **dependency health**
- Alerts on user symptoms, each with a runbook
- Critical dependencies: health check, alert, owner, escalation

HEALTH CHECKS

- **Application:** health endpoint, errors, response time
- **Database:** connections, locks, replication lag, space
- **Infrastructure:** CPU, memory, disk, network
- **Jobs:** success, duration, missed runs
- **Dependencies:** DB, external APIs, DNS, load balancers, queues, identity provider, storage, network

SERVICE LEVELS

- **SLI** — what we measure (availability, latency, error rate)
- **SLO** — internal target, e.g. 99.9% < 300 ms
- **SLA** — contract with the business, looser than SLO
- Targets follow criticality tier; breaches reviewed
- Also tracked: MTTR per tier, MTBF (Mean Time Between Failures)

BACKUP + VERIFIED RESTORE

- Backups monitored: success, size, duration
- **Restore tested regularly** in an isolated environment
- Measured restore time vs RTO
- A backup never restored is not a backup

DISASTER RECOVERY

- **RTO** — how long we may be down
- **RPO** — how much data we may lose
- DR plan, failover and failback procedure
- DR test at least yearly for Tier-1

SECURITY

- Patching cadence (e.g. monthly, critical within days)
- Vulnerability management: scan, triage, fix by SLA
- Certificate lifecycle: register, expiry alerts, auto-renew
- Access review, audit logs

CAPACITY & PERFORMANCE

- Trends: CPU, memory, storage, DB growth, queues
- Forecast vs limits; plan 3–6 months ahead
- Performance baselines and regression checks
- Cost: cloud spend, licences, contracts

CONTINUOUS IMPROVEMENT

- Inputs: incidents, PIRs, SLO breaches, alerts, tech debt, user feedback
- One prioritised improvement backlog
- Monthly service review with owner: SLOs, MTTR trend, risks
- Feeds back into sections 1, 5 and 7

DELIVERABLES

- ▶ Dashboards & alert catalogue
- ▶ Runbooks, KB, KEDB
- ▶ SLO definitions & monthly report
- ▶ Backup & restore test records
- ▶ DR plan & test report
- ▶ Patch, vulnerability & certificate reports
- ▶ Capacity report
- ▶ Improvement backlog

EXAMPLE

Quarterly restore test of the core database into an isolated environment: restore took 3 h 40 min against an RTO of 4 h. Pass, but close — action: faster backup storage and parallel restore, retest next quarter.

EXIT CRITERIA (ONGOING)

SLOs met or every breach has an action · restore verified within the agreed period · DR tested · no critical vulnerability past its deadline · no certificate expiring unnoticed · improvement backlog reviewed monthly.



Cross-cutting: knowledge transfer & observability

Runs through the whole framework

Taking over an unknown IT project — playbook · p.14

Knowledge transfer

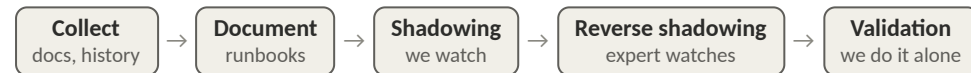
Collect in 2 · plan & own in 6 · keep alive in 10 · track as risk in 7

Risk: “Only one person knows how this works.” Treat it as a risk in the register, with owner, mitigation and date.

WHAT TO COLLECT

- Existing documentation
- Runbooks
- Architecture documentation
- Known issues
- Previous incidents
- Troubleshooting procedures
- Operational procedures
- Deployment procedures
- Vendor documentation
- Tribal knowledge (in people's heads)

HOW TO TRANSFER



- Validation = real proof: restore a DB, deploy a release, run an incident drill without help
- Sign-off by the SME and the service owner

KNOWLEDGE-TRANSFER PLAN (EXAMPLE)

Week	Activity	Proof of knowledge
1	Collect docs, incidents, runbooks; walkthrough with SME	Gap list agreed
2–3	Shadowing: deploy, restart, restore	Runbooks written by the new team
4–5	Reverse shadowing: new team operates, SME watches	No SME intervention needed
6	Validation: restore test and incident drill alone	Sign-off by SME and service owner

KNOWLEDGE TRANSFER IS COMPLETE WHEN

- ✓ Runbooks exist for routine and emergency procedures
- ✓ New team handled a real incident without the SME
- ✓ Restore and deployment performed by the new team
- ✓ Known issues and workarounds are in the KEDB
- ✓ Architecture docs reviewed and current
- ✓ Vendor contacts and contract terms known by the team

Observability

Health now in 0 · coverage in 2 · gaps as risks in 7 · validation in 9 · operate in 10

How do we know the service is broken?

DETECT

- Health checks
- Synthetic monitoring
- Availability & error-rate SLIs
- Alerts on user-facing symptoms

How do we determine why it is broken?

DIAGNOSE

- Metrics: saturation, resources
- Logs: errors with context
- Traces: which call, which service
- Dashboards & recent changes

Is the problem in our service or in a dependency?

ISOLATE — DEPENDENCY MONITORING

- Database availability
- External APIs
- DNS
- Load balancers
- Message queues
- Authentication / identity provider
- Storage
- Upstream & downstream services
- Network connectivity

A service can be healthy internally while unusable because one of its dependencies is failing. Dependency health belongs on the dashboard; every critical dependency has a health check, alert, owner and escalation path.

SIGNALS & MEASURES

Signals: metrics · logs · traces · alerts · dashboards · health checks · synthetic monitoring

Measures: availability · performance (latency) · error rates · capacity

Alert on symptoms, diagnose with causes. Every alert is actionable and has a runbook.

OBSERVABILITY COVERAGE CHECK

- ✓ Every Tier-1 service has a health check and a synthetic test
- ✓ Availability, latency and error-rate SLIs defined and on a dashboard
- ✓ Logs centralised, structured, searchable, PII masked
- ✓ Traces across service boundaries for critical flows
- ✓ Critical external and internal dependencies are monitored
- ✓ Dependency failures can be distinguished from application failures
- ✓ Batch jobs monitored: success, duration, missed runs
- ✓ Alerts reviewed monthly: remove noise, close gaps
- ✓ Monitoring of the monitoring: collectors and agents alive



First 30 days taking over an unknown IT service

Practical checklist

Taking over an unknown IT project — playbook · p.15

Day 1	First 3 days	First week	First 2 weeks	First 30 days
☐ Check production health ☐ Check active incidents (P1/P2) ☐ Confirm service owner & escalation contacts ☐ Get critical access: PROD read, monitoring, ticketing, vault ☐ Find monitoring & dashboards ☐ Check backup status ☐ Check certificates expiring < 30 days	☐ Burning-issues list with owners ☐ Review last 3 months of incidents & changes ☐ Meet owner, ops teams, service desk, vendors ☐ Collect existing documentation & runbooks ☐ Understand change process & deployment basics ☐ Decide on change freeze if needed	☐ Complete inventory ☐ Map dependencies & data flows ☐ Environment matrix ☐ Identify critical risks, start register ☐ Map stakeholders & escalation paths ☐ Validate monitoring coverage ☐ Validate backups ☐ Understand deployment end-to-end	☐ Criticality tiers agreed ☐ Access matrix; missing access fixed ☐ Ownership matrix & RACI drafted ☐ Knowledge transfer started (shadowing) ☐ Scope & priorities drafted ☐ Working agreement: workflows, cadence ☐ Alert review: noise vs gaps	☐ Ownership established ☐ Documentation baseline created ☐ Critical risks addressed or mitigated ☐ Runbooks for top alerts & procedures ☐ Monitoring improved, SLIs defined ☐ Restore tested and verified ☐ Release process understood & documented ☐ Improvement backlog created & prioritised ☐ First service review with owner

Rule of thumb: **stabilize** → **understand** → **own** → **protect** → **improve**. Don't redesign anything before the inventory, owners and a verified restore exist.